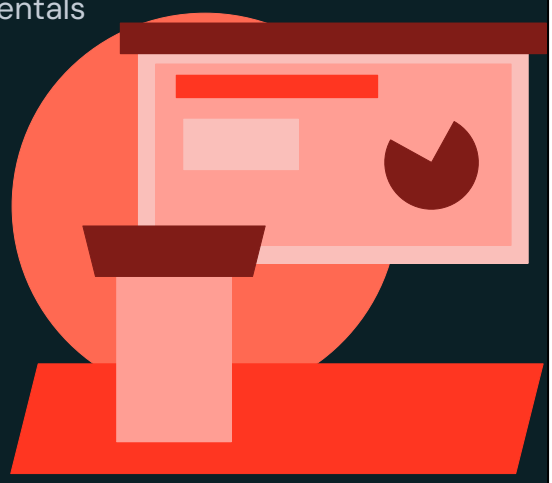
databricks

Lakeflow Spark Declarative Pipeline Fundamentals

LECTURE

Common Pipeline Settings

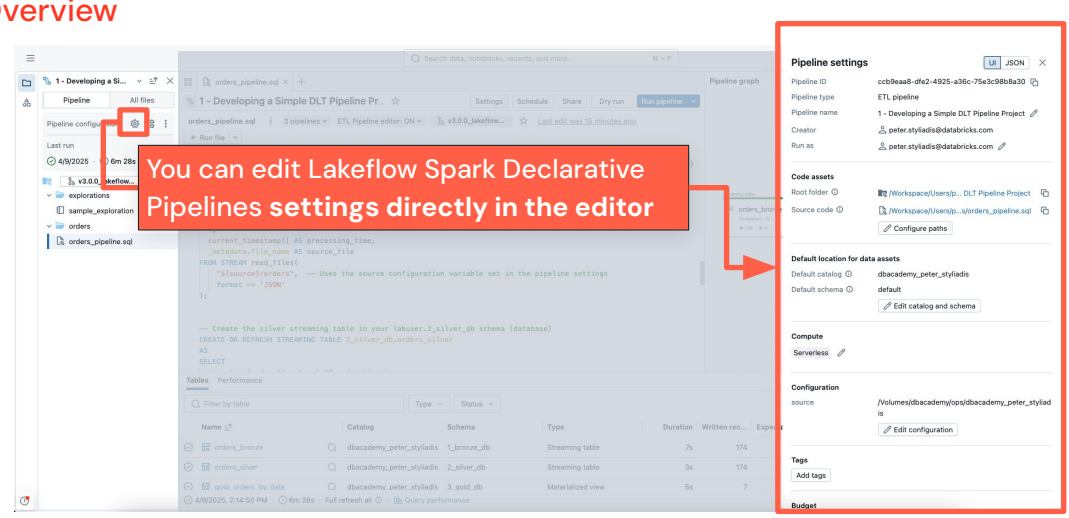


© Databricks 2025. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the Apache Software Foundation.

In this lecture, you will learn about the various settings available in Declarative Pipelines and how to configure them to customize your pipeline's behavior.

Common Pipeline Settings

Overview



© Databricks 2025. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the Apache Software Foundation.

Declarative Pipelines come with a variety of settings you can easily configure to customize how your pipeline behaves.

With the new multi-file editor, accessing and editing these settings is simple and intuitive. Just click the small gear icon in your editor window to quickly open the pipeline settings popup. No need to leave your development environment.

From there, you can adjust key settings and options directly, helping you tailor your pipeline to your specific requirements without interrupting your workflow.

Common Pipeline Settings

Compute

Code Assets

Configuration
(parameters)

© Databricks 2025. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the Apache Software Foundation.

There are many pipeline settings you can customize, but three of the most common you'll work with are:

- **Compute** - Choose the resources and environment your pipeline will run on.
- **Code Assets** - Manage the files and code modules that make up your pipeline.
- **Configuration** (Parameters) - Set pipeline parameters to control behavior dynamically during execution.

We'll focus on these three settings now, but we'll explore additional options during the upcoming demonstration.

Common Pipeline Settings

Compute

Code Assets

Configuration
(parameters)

SERVERLESS

- Databricks **recommends** developing new pipelines using serverless
- **Optimizes** cost while **maintaining** performance
- **Enables** you to focus on code rather than managing infrastructure
- Optional **Serverless Performance optimized** setting for time-sensitive workloads

- Serverless compute includes **Incremental refresh for materialized views**
- A cost-based optimizer to power fast and efficient transformations for materialized views on Serverless compute

© Databricks 2025. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the Apache Software Foundation.

Let's begin with Compute, which lets you select the resources and environment where your pipeline will run.

The recommended option is **Serverless** compute.

- Databricks advises developing new pipelines using Serverless, as it simplifies pipeline management.
- Serverless optimizes costs while maintaining strong performance.
- It allows you to focus fully on your code without needing to manage or provision infrastructure.
- For time-sensitive workloads, there's an optional Serverless Performance Optimized setting to boost responsiveness.

Additional benefits of **Serverless** compute include:

- Support for incremental refresh of materialized views.
- A cost-based optimizer that enables fast and efficient transformations of materialized views, improving pipeline efficiency and speed.

Refer to: Configure a serverless pipeline | Databricks on AWS

Common Pipeline Settings

Compute | Code Assets | Configuration (parameters)

SERVERLESS

- Databricks **recommends** developing new pipelines using serverless
- **Optimizes** cost while **maintaining** performance
- **Enables** you to focus on code rather than managing infrastructure
- Optional **Serverless Performance optimized** setting for time-sensitive workloads

CLASSIC (fixed size)

- Users need **permission** to create compute for Declarative Pipelines
- Workspace admins **can configure cluster policies** to provide users with access to compute resources for Declarative Pipelines

AUTOSCALING

- Enhanced autoscaling **is enabled by default** for all new pipelines.
- **Optimizes cluster utilization** by automatically allocating cluster resources based on workload volume

© Databricks 2025. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the Apache Software Foundation.

In addition to Serverless, there is also **Classic** Compute, which uses fixed-size clusters.

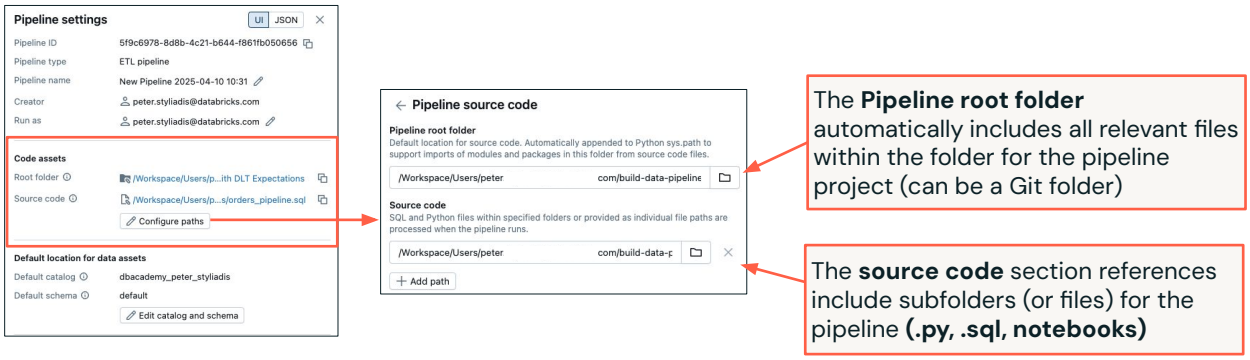
- Users need the appropriate permissions to create compute resources for Declarative Pipelines using Classic Compute.
- Workspace administrators can set up cluster policies to control and provide access to compute resources for users working with Declarative Pipelines.

Another important feature is **Autoscaling**:

- Enhanced autoscaling is enabled by default for all new pipelines using Classic Compute.
- This automatically adjusts cluster size based on workload volume, optimizing resource utilization and helping control costs without manual intervention.

Common Pipeline Settings

Compute | Code Assets | Configuration (parameters)



© Databricks 2025. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the Apache Software Foundation.

Now, let's talk about Code Assets in Declarative Pipelines.

This setting controls what code files your pipeline uses during execution.

- The **Pipeline Root Folder** is set to automatically and includes all relevant files within that folder for your pipeline project (unless otherwise specified by the user). This can be a Git folder, enabling easy version control and collaboration.

- The **Source Code Section** lets you specify which subfolders or individual files to include in your pipeline, typically sub folders and files within the root folder. These can be Python scripts, SQL files, notebooks, and more.

Together, these settings ensure your pipeline runs with the exact code it needs, organized and versioned as part of your project.

Common Pipeline Settings

Compute | Code Assets | Configuration (parameters)

A pipeline's configuration is **a map of key value pairs** that can be used to parameterize your code:

- Improve code readability/maintainability
- Reuse common parameters in multiple pipelines files

```
CREATE STREAMING TABLE bronze
SELECT *
FROM STREAM read_files(
  "${source}/orders",
  format => 'JSON');
```

In SQL reference the key's value using the **${key-name}** syntax

© Databricks 2025. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the Apache Software Foundation.

Finally, let's cover Configuration, also known as parameters.

A pipeline's configuration is a collection of key-value pairs that help you parameterize your code. This approach brings several benefits:

- Improves code readability and maintainability by centralizing key values.
- Enables you to reuse common parameters across multiple pipeline files, avoiding hardcoding values.

For example, you might have a parameter called **source** that points to a specific volume path.

In your SQL code, you can reference this parameter using the syntax **${source}**, which dynamically inserts the value at runtime.

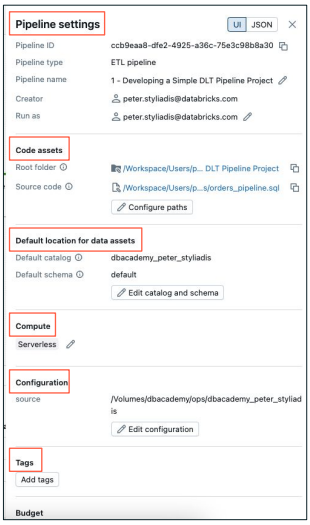
This makes your pipeline flexible and easier to update without changing code in multiple places.

Common Pipeline Settings

Common Settings

There are **various other settings** we will explore throughout the course, including:

- Pipeline settings
- Code assets
- Default location for data assets
- Compute
- Configuration
- Environment
- Tags
- Budget
- Advanced settings



© Databricks 2025. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the Apache Software Foundation.

Beyond the core settings we've discussed, there are several other pipeline settings you'll encounter throughout the course.

These include:

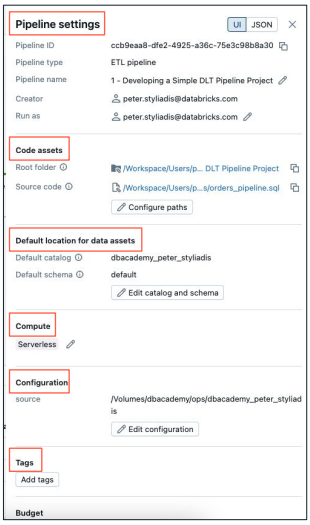
- Pipeline Settings - General pipeline behavior and metadata
- Code Assets - Managing your pipeline code files
- Default Location for Data Assets - Setting where data is stored by default
- Compute - Choosing your execution environment.

Common Pipeline Settings

Common Settings

There are **various other settings** we will explore throughout the course, including:

- Pipeline settings
- Code assets
- Default location for data assets
- Compute
- Configuration
- Environment
- Tags
- Budget
- Advanced settings



© Databricks 2025. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the Apache Software Foundation.

- Configuration - Parameterizing your pipeline code
- Environment dependencies - A dependency is a line in a pip requirements file.
- Tags - Organizing and categorizing pipelines for easy management
- Budget - Defining cost controls and limits
- Advanced Settings - Additional options for fine-tuning your pipeline

We'll explore these in more detail with hands-on demonstrations to help you get the most out of Spark Declarative Pipelines.

© Databricks 2025. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the Apache Software Foundation.

Thank you for completing this lesson and continuing your journey to develop your skills with us.